

Controle de Obra

Sistema de gestão financeira de obras

Documentação completa: problema, solução e funcionalidades
MVP - Fase 1 - versão 1.0

1. Problema

Durante uma obra, os gastos acontecem de forma fragmentada e se repetem para os mesmos materiais (cimento, areia, brita). Uma planilha comum permite registrar valores, mas obriga a pessoa a somar, comparar e interpretar tudo manualmente. O fluxo real é: *gastei, registrei, somei, descobri quanto sobrou*.

Isso dificulta responder, com rapidez e segurança, às perguntas que orientam as decisões:

- Quanto já gastei e quanto ainda tenho disponível?
- Qual é a minha meta de gasto e quanto dela já consumi?
- Com o que estou gastando e qual categoria consome mais?
- Quanto é gasto recorrente e quanto é pontual?
- Quanto cada item já consumiu, somando todas as compras?
- A obra está dentro do limite financeiro?

Há ainda um problema de qualidade dos dados: nomes de itens inconsistentes, ausência de datas e de classificação, e nenhuma validação. O resultado é uma visão financeira lenta de obter e sujeita a erros de cálculo.

2. Solução desenvolvida

O **Controle de Obra** transforma o controle manual em uma aplicação orientada a dados: lançamentos estruturados, cálculos automáticos e um painel que mostra a situação financeira em poucos segundos. O fluxo passa a ser: *tenho um orçamento, acompanho o que já gastei, entendo onde estou gastando, acompanho minha meta e tomo decisões melhores*.

O sistema nasce **zerado** e não depende de nenhuma planilha. Dados externos entram pela **importação de arquivos CSV**, com mapeamento de colunas e validação, e o histórico pode ser exportado no mesmo formato. Os dados ficam em um banco SQLite local.

Princípios: simplicidade (não exige conhecimento contábil), foco financeiro, informação acionável, automatização dos cálculos e capacidade de evolução sem reconstrução.

3. Funcionalidades

Cada funcionalidade é descrita com sua função e o objetivo que atende.

Funcionalidade	Descrição	Objetivo
Cadastro da obra (aba Obra)	Define nome, orçamento, meta de gasto, data de início, previsão de conclusão e status. Valida: nome obrigatório, orçamento e meta maiores que zero, datas coerentes e meta acima do orçamento somente se a opção for marcada.	Estabelecer os limites financeiros que servem de base para todos os indicadores, separando orçamento disponível de meta de gasto.
Categorias	Cria, renomeia e desativa categorias (8 sugeridas: Materiais, Mão de obra, Hidráulica, Elétrica, Estrutura, Equipamentos, Serviços, Outros). Nunca são apagadas.	Agrupar itens para análise e preservar a integridade do histórico.
Itens	Cadastra o que é comprado ou pago, com categoria, comportamento (recorrente ou pontual), unidade e status ativo/inativo. Mostra o consolidado de cada item (lançamentos, quantidade e total).	Padronizar nomes, evitar duplicidades e saber quanto cada item consumiu.

Funcionalidade	Descrição	Objetivo
Registro de gasto (+ Novo gasto)	Formulário com data, item, quantidade, valor unitário e observação. O total é calculado automaticamente (quantidade x valor unitário). Itens novos são criados na hora. Confirma ao salvar.	Registrar despesas com poucos toques e sem cálculo manual.
Atualização automática	Após registrar, editar ou excluir, todos os indicadores são recalculados a partir dos lançamentos.	Garantir consistência: o painel nunca diverge do histórico.
Painel (dashboard)	Mostra orçamento, meta, total gasto, saldo, restante da meta, % do orçamento, % da meta, total recorrente, total pontual, gastos sem classificação, barra recorrente x pontual x sem classificar, gasto por categoria e os 10 maiores itens.	Responder "como está financeiramente minha obra?" em poucos segundos.
Alertas visuais	Classifica o consumo do orçamento e da meta em Dentro do limite (até 90%), Próximo do limite (90% a 100%) e Ultrapassou (acima de 100%).	Antecipar riscos antes de estourar os limites.
Histórico e filtros	Lista data, item, categoria, comportamento, quantidade, valor unitário e total. Ações: ver (detalhes e origem), editar e excluir (com confirmação). Filtros combináveis: busca, item, categoria, recorrente/pontual e período.	Consultar, auditar e corrigir lançamentos.
Importação de CSV	Lê .csv/.txt (separador ; , ou tab; números 1.234,56 ou 1234.56; datas AAAA-MM-DD ou DD/MM/AAAA). Detecta colunas, permite ajustar o mapeamento, mostra prévia e total, aponta linhas inválidas e ignora linhas idênticas a importações anteriores. Cria categorias e itens inexistentes.	Alimentar o sistema com dados externos (planilhas, outros sistemas) sem digitação.
Exportação de CSV	Gera um CSV com todos os gastos, itens e categorias, no formato aceito pela importação.	Permitir análise externa e backup dos dados.
Autoteste de cálculos (aba Obra)	Executa casos de teste dos cálculos financeiros (exemplo do plano, sem gastos, gasto igual ao orçamento e acima da meta).	Dar confiança de que as fórmulas estão corretas.
Apagar todos os dados	Zera obra, itens, categorias e gastos, com confirmação em dois cliques.	Reiniciar o sistema para uma nova obra.

4. Regras de negócio

Regra	Fórmula
RN-001 Total gasto	soma dos valores totais de todos os gastos
RN-002 Saldo	orçamento - total gasto (pode ser negativo)
RN-003 % do orçamento	total gasto / orçamento x 100
RN-004 Restante da meta	meta - total gasto (negativo = meta ultrapassada)
RN-005 % da meta	total gasto / meta x 100
RN-006 / RN-007	soma dos gastos de itens recorrentes / pontuais
RN-008 Categoria	soma dos gastos dos itens da categoria

A data é obrigatória em gastos lançados manualmente; linhas importadas sem data são aceitas e exibidas como "sem data". Valores nunca são estimados: sem categoria ou comportamento, o gasto aparece como "Sem categoria" ou "Sem classificar".

5. Arquitetura e organização de arquivos

Aplicação local em três camadas: interface (HTML, CSS e JavaScript separados), API REST em Python e banco SQLite. Não há dependências além do Python 3.8+.

controle-obra/	
server.py	servidor HTTP + API REST + acesso ao SQLite
data/obra.db	banco de dados (criado vazio no 1o uso)
public/index.html	estrutura da página
public/css/style.css	estilos (tema claro/escuro, responsivo)
public/js/app.js	interface, painel, histórico, importação CSV
docs/documentacao.pdf	esta documentação
exemplo_importacao.csv	arquivo de exemplo para importar
README.md	instruções rápidas

Como executar: na pasta do projeto, rode `python3 server.py` e abra `http://localhost:8000`. O servidor escuta apenas em 127.0.0.1 (acesso local). Outra porta: `python3 server.py 8080`.

6. Banco de dados

SQLite com integridade referencial ativa. Relacionamentos: uma obra tem N gastos; uma categoria tem N itens; um item tem N gastos.

Tabela	Campos principais	Regras
obra	id (=1), nome, orcamento, meta, inicio, fim, status, meta_alem	Registro único; orçamento 0 = obra não configurada
categorias	id, nome, ativo, criado	Nome único (sem diferenciar maiúsculas); sem exclusão
itens	id, nome, categoria_id, comportamento, unidade, ativo	Nome único; comportamento: recorrente ou pontual
gastos	id, item_id, data, quantidade, valor_unitario, valor_total, observacao, origem, criado, atualizado	quantidade > 0; valor_unitario >= 0; valor_total calculado no servidor; origem = manual ou csv:arquivo

7. API REST

Rota	Função
GET /api/state	Retorna obra, categorias, itens e gastos
PUT /api/obra	Cria/atualiza a obra (com validações)
POST /api/categorias, PUT /api/categorias/{id}	Cria, renomeia, ativa ou desativa categoria
POST /api/itens, PUT /api/itens/{id}	Cria item; altera categoria, comportamento, unidade e status
POST /api/gastos, PUT DELETE /api/gastos/{id}	Registra, edita ou exclui gasto
POST /api/import	Importa linhas (JSON) em lote, com detecção de duplicadas
GET /api/export	Baixa o histórico em CSV
POST /api/reset	Zera todos os dados

Todas as validações são aplicadas no servidor; a interface apenas antecipa as mensagens. Erros retornam código HTTP 4xx com a descrição do problema.

8. Formato do arquivo CSV

Primeira linha com os nomes das colunas. Exemplo:

```
item;quantidade;valor_unitario;data;categoria;comportamento;unidade;observacao  
Cimento;10;35,50;15/03/2026;Materiais;recorrente;saco;Loja A  
Portão;1;1800;20/03/2026;Estrutura;pontual;un;
```

Obrigatórias: **item**, **quantidade**, **valor unitário**. Se um item já existir, a categoria e o comportamento do CSV não o alteram. O arquivo `exemplo_importacao.csv` traz 48 lançamentos (sem data) para teste.

9. Fora do escopo e evolução

Ficam para fases futuras: planejamento e previsão de custo final, orçamento por categoria, fornecedores, estoque, notas fiscais, anexos, alertas avançados e histórico de alterações. O modelo de dados já separa obra, categoria, item e gasto para permitir essa evolução sem reconstrução. Autenticação não faz parte desta versão, pois o servidor é de uso local.

10. Publicação em produção (Vercel)

Atualização: em produção o sistema usa **PostgreSQL** (Neon, via Vercel Storage) no lugar do SQLite descrito nas seções 5 a 7, e passa a exigir **senha de acesso**. Em uso local continua valendo o SQLite, sem senha.

Arquitetura de produção

Camada	Local	Produção (Vercel)
Interface	public/ servida pelo server.py	public/ servida como site estático
API	server.py chama api/_core.py	api/index.py (função serverless) chama api/_core.py
Banco	SQLite em data/obra.db	PostgreSQL (variável DATABASE_URL); tabelas criadas no 1o acesso
Acesso	Sem senha, só em localhost	Login por senha (APP_PASSWORD), cookie assinado HttpOnly/Secure por 7 dias

Passo a passo

- Enviar o projeto ao GitHub (api/, public/, vercel.json e requirements.txt na raiz).
- Na Vercel: Add New > Project, escolher o repositório, Framework Preset "Other", sem Build Command.
- Aba Storage: criar um banco Neon (Postgres) e conectá-lo ao projeto; a variável DATABASE_URL é criada sozinha.
- Em Environment Variables: definir APP_PASSWORD (obrigatória) e APP_SECRET (texto aleatório longo).
- Fazer Redeploy, abrir o endereço, entrar com a senha e criar a obra.

Novas rotas e proteções

Item	Descrição
GET /api/session	Informa se o acesso é protegido e se a sessão está ativa
POST /api/login, POST /api/logout	Abre e encerra a sessão (senha comparada em tempo constante; resposta atrasada em 1 s em caso de erro)
Demais rotas /api/*	Retornam 401 sem sessão válida
Escritas (POST/PUT/DELETE)	Exigem Content-Type application/json (proteção contra requisições de outros sites)
Configuração ausente	Sem APP_PASSWORD ou sem banco, a API responde 503 explicando o que falta, em vez de operar sem proteção ou perder dados

Backup e limitações: o histórico pode ser exportado em CSV (aba Importar) e o Neon oferece restauração pontual. Há uma única senha compartilhada (sem usuários individuais) e não há limite de tentativas de login além do atraso de 1 segundo; usuários individuais e histórico de alterações ficam para fases futuras.